深圳大学 SHENZHEN UNIVERSITY

课程 COURSE

Big Data Analytics
大数据分析 [2026 国际商务]

Lecture 15 — Big Data in Marketing 第十五讲·营销中的大数据
测量、选择、反应与客户价值

任课教师：戴谢尔 (Xieer Dai, David)

深圳大学 Shenzhen University

·第 3–14 周 · 星期四 11-12 节

上课地点：四方楼南 209



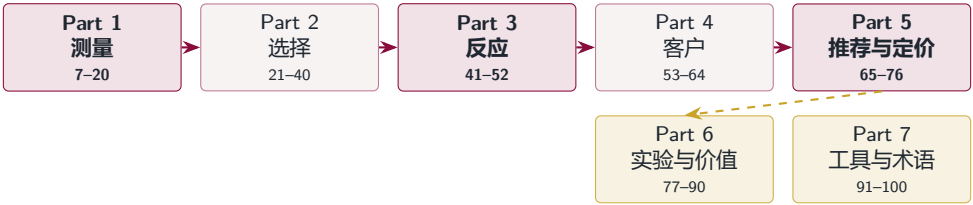
学习目标 Learning Objectives

完成本讲后，你应能够：

1. 设计一份可信的量表问卷并检验信度效度；
2. 用联合分析 conjoint analysis 估出属性效用与重要性；
3. 用logit 选择模型把效用变成市场份额；
4. 估特征价格 hedonic 与响应曲线，算清价格弹性与最优价；
5. 做漏斗、归因、流失与 CLV，并用A/B 与功效验证效果；
6. 说清隐私新政对营销数据的影响，写出合规的测量方案。

来源：对应课程大纲第 15 讲目标：掌握营销数据采集与实验设计；能用联合分析、推荐与客户价值模型支撑决策。

本讲路线图 Roadmap（100 帧）



主线：测量 → 选择 → 反应 → 客户 → 推荐定价 → 实验验证

代码页 10 帧•样本页 14 帧；中文案例，术语全程中英对照

与前面九讲的衔接 From Toolkit to Marketing

讲次	给了什么	在本讲变成什么
L2 / L5	抽样、问卷、实验设计	量表与 A/B（帧 11、84）
L6 / L7	分类与回归	响应模型、流失、弹性（帧 46、78、71）
L8	聚类与降维	客户分群与 RFM（帧 54）
L9 / L10	深度与文本	推荐、多模态内容（帧 68、89）
L12（大纲）	因果推断	增量、MMM（帧 86、88）

营销不是新方法，而是把已有方法用在" 人" 的数据上。本讲每个方法都配一个" 能算出来的数"，你可以直接搬进项目。

来源：衔接为本讲组织；L12 因果推断不在本课件的讲义序列内，按大纲仅作方法引用。

本讲输出物 Deliverables

1) 量表与设计

问卷题项表、信度效度检验、conjoint 属性–水平表与正交设计。

2) 11 个程序

信度、conjoint、logit、hedonic、漏斗归因、推荐、弹性、流失、CLV、A/B、术语。

3) 术语表

85 条中英对照（CSV + Word/Excel），按六个主题分组。

所有数字都来自可复现的程序：`website/code/lecture15/`，种子固定 20261216；数据全部合成（问卷 600 份、选择 800 次、房价 1,200 套、点击流 6,000 会话、评分矩阵 500×60、A/B 各 5,000）。

来源：程序与数据随课程仓库发布。

全讲导览 Map of the Lecture

部分	帧	关键问题
营销数据与测量	7–20	测得到底准不准？问卷怎么设计？
联合分析与离散选择	21–40	属性值多少？人会选哪个？
特征价格与响应模型	41–52	价格由什么决定？投放多少划算？
消费者数据与归因	53–64	客户怎么走？功劳算给谁？
推荐、定价与促销	65–76	推荐什么？定多少钱？怎么打折？
流失、CLV 与实验	77–90	谁会走？值多少？怎么验证？
工具与术语	91–100	用什么工具？术语怎么背？

读法：每个方法都走一遍问题 → 设计 → 估计 → 样本图 → 怎么用。

来源：导览为本讲安排。

营销分析的四类问题 Four Questions

问题	典型问法	方法	本讲帧
谁买	客户长什么样、会走吗	分群、流失模型	54、78
买什么	偏好哪种配置	联合分析、推荐	21、65
为什么	广告有没有用	归因、A/B、增量	58、84
值多少	这个客户值多少钱	CLV、价格弹性	80、71

先想清楚问的是哪一类，再决定收什么数据——问题决定数据结构。这是营销分析的第一个分岔口。

来源：四类问题为本讲组织，依据营销分析的标准分工（描述、偏好、效果、价值）。

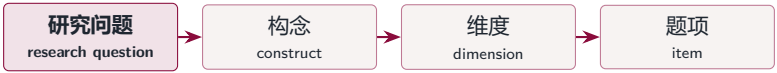
五类营销数据 Five Data Sources

来源	典型内容	主要偏误	成本
问卷	态度、意图、人口统计	自报偏差、样本偏差	中
交易/CRM	购买记录、金额、频次	只看到"买过的人"	低
点击流	曝光、点击、路径	埋点缺失、机器人	中
社交媒体	内容、互动、口碑	平台口径、水军	中
实验	随机分组的对照	溢出、新奇效应	高

五种数据的最佳组合是"态度（问卷）+ 行为（交易/点击）+ 因果（实验）"：问卷说"为什么"，行为说"做了什么"，实验说"是不是真的有用"。

来源：五类来源与偏误据营销研究实践；组合建议为本讲补充。

问卷设计（一）：从研究问题到题项



例：品牌信任

构念 = 品牌信任；维度 = 能力、善意、诚实；题项 = ”这个品牌不会让我失望”“我相信它的质量”。

一个构念要几条题项

通常 3–5 条：太少测不稳，太多让受访者疲劳。本讲用 4 条（帧 14 的信度就是靠它）。

最常见的错误：直接从文献抄题项，却没说清它对应哪个构念的哪个维度。

来源：构念—维度—题项的操作化据 Churchill (1979) 的量表开发范式。

问卷设计（二）：题项类型与措辞陷阱

四种题项

- 李克特：1–5 同意程度；
- 排序：给偏好排序；
- 选择：从几个方案里挑一个（帧 30）；
- 开放：一段文字（衔接 L10）。

四个措辞陷阱

1. 双重问题：" 服务好且价格低吗"；
2. 引导性：" 您也认为贵得不合理吧"；
3. 社会赞许：少报吸烟、多报读书；
4. 默认选项：不设" 不知道" 会逼出答案。

自检：把每个题项读给同事听，问" 你怎么答"——如果两个人理解不同，题项就要改。

来源：四种题项与四类陷阱据问卷设计实践（Dillman 的 Tailored Design 方法）。

抽样：概率与非概率 Sampling

方式	怎么做	能不能推断总体
简单随机	从名单里等概率抽	能
分层	按层分别抽再合并	能，且更精确
整群	抽群再普查群内	能，成本低
配额	按比例凑够人数	不能（非概率）
便利	谁方便找谁	不能

样本量：估计比例时 $n \approx 1.96^2 p(1-p)/e^2$——$p = 0.5$、误差 3% 时约需 1,067 份。营销项目里更常见的问题是样本偏年轻、偏一线城市，而不是量不够。

来源：抽样方式与样本量公式据标准抽样理论；营销样本偏差的观察为本讲补充。

执行方式与模式效应 Mode Effects

四种方式

面访（深但贵）、电话（快但短）、网络（便宜但偏）、混合（互补但难比）。

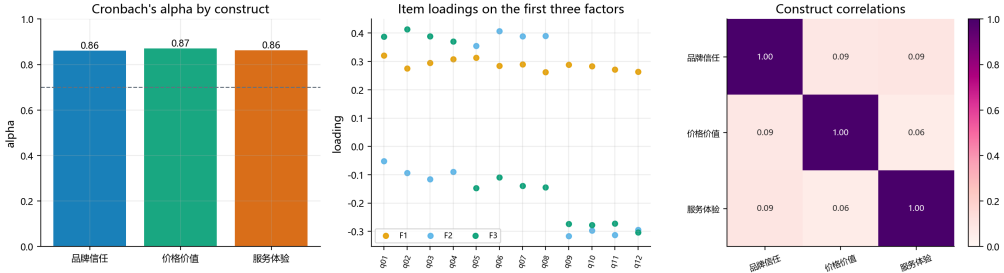
模式效应

同一道题，面访会被社会赞许压低负面回答；网络问卷更愿意说真话，但样本更年轻。

所以换执行方式就等于换了一把尺子——如果两轮调查的方式变了，趋势比较就要打折扣。这一点要写进方法说明。

来源：模式效应据调查方法学文献；实践提醒为本讲补充。

【样本 + 可视化】一份营销问卷的骨架与三个构念



左：三个构念（品牌信任、价格价值、服务体验）的Cronbach's α，都高于 0.7 的门槛；中：12 个题项在前三个因子上的载荷——每个构念的题项各自成群；右：三个构念得分的相关矩阵，组间相关很低（区分效度成立）。

程序 `l15_01_survey_reliability.py` ·样本 `l15_survey.csv`（600 位受访者、12 个题项）

信度：同一样东西测两次像不像 Reliability

三种信度

1. 内部一致性：Cronbach's α；
2. 重测信度：隔两周再测一次；
3. 分半信度：奇偶题分开算。

α 的公式与陷阱

$$\alpha = \frac{k}{k-1}\left(1 - \frac{\sum \sigma_i^2}{\sigma_T^2}\right).$$

α 会随题项数上升：20 条烂题也能凑出 0.8。

本讲实测：三个构念的 α 在 0.72–0.81，整表 0.761。判读惯例：> 0.7 可用，> 0.8 较好；但一定要报题项数与构念定义。

来源：α 公式与判读据 Cronbach (1951) 与实践惯例。

效度：测的是不是那个东西 Validity

类型	问的问题	怎么查
内容效度	题项覆盖全部维度吗	专家评审、预测试
构念效度	真的测到抽象构念吗	因子分析（帧 16）
收敛效度	同构念不同量表相关吗	与既有量表的相关
区分效度	不同构念分得开吗	相关系数小于 $\sqrt{\text{AVE}}$
效标效度	能预测该预测的吗	与购买行为的相关

一句口诀：信度是必要条件，效度是充分条件——α 高只说明" 测得稳"，不说明" 测对了"。

来源：五类效度据 Campbell & Fiske (1959) 与量表开发实践。

【代码】信度、探索性因子分析与区分效度

```python
def cronbach_alpha(frame):                              # reliability of a block of items
    k = frame.shape[1]                                  # number of items
    var_items = frame.var(axis=0, ddof=1).sum()         # sum of item variances
    var_total = frame.sum(axis=1).var(ddof=1)           # variance of the total score
    return float(k / (k - 1) * (1 - var_items / var_total)) # the standard formula
rows = []                                               # one row per construct
for name, cols in blocks.items():                       # loop over the constructs
    alpha = cronbach_alpha(survey[cols])                # reliability
    rows.append({"construct": name, "items": len(cols), "alpha": round(alpha, 3), # the row
                 "mean": round(mean, 2)})               # more columns
X = StandardScaler().fit_transform(survey[items])       # standardise the twelve items
pca = PCA(n_components=4).fit(X)                        # four components
loadings = pd.DataFrame(pca.components_.T, index=items, # item loadings
                        columns=["F1", "F2", "F3", "F4"]) # four factor columns
corr = scores.corr()                                    # construct correlations
sqrt_ave = float(np.sqrt(avg_var))                      # square root of the average variance
```

实测：$\alpha = 0.72$–0.81，整表 0.761；四个成分解释方差 0.61（前两个为主）；构念间相关 0.06–0.09，远小于 $\sqrt{\text{AVE}}$，区分效度成立。完整程序 115_01。

测量误差与共同方法偏差

两类误差

随机误差：让估计不稳（可以平均掉）；系统误差：让估计有偏（平均不掉，最危险）。

共同方法偏差

同一份问卷、同一个人、同一时间测所有变量——变量之间的相关可能只是方法造成的。

三种补救：时间上分开测（前后测）、来源上分开（自评 + 客观记录）、统计上检验（Harman 单因子：第一个因子解释不到一半，就还好）。

来源：共同方法偏差与 Harman 检验据 Podsakoff et al. (2003)。

非响应与加权 Non-response

三种无回答

- 单元无回答：整份问卷没交；
- 项目无回答：某题跳过；
- 覆盖误差：名单里根本没有某类人。

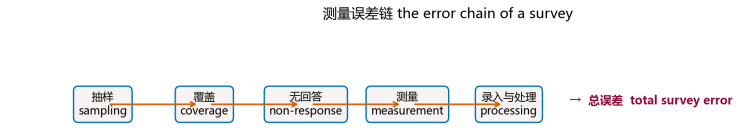
两种处理

加权：按人口结构给样本加权；插补：用其他变量补缺失（要说明方法）。

判断有没有偏：比较回答者与未回答者在已知变量上的差异（年龄、城市、历史消费）——本讲没有真实未回答者，所以这一步只能靠你项目里的数据做。

来源：无回答类型与加权据调查方法学；判断方法为本讲补充。

【样本 + 可视化】测量误差的链条



本讲只改得了最后两个环节；前三个决定样本能不能代表总体。

从抽样到处理，每一环都会引入误差，合起来叫总调查误差 total survey error。本讲能改的只有最后两环（测量与处理）；前三个环节决定样本能不能代表总体——所以帧 11 的抽样设计与帧 18 的无回答分析，比模型选择更重要。

程序 `l15_01_survey_reliability.py` ·图为本讲自制

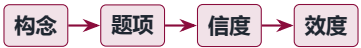
小结：测量决定上限

三句话

构念要操作化成题项；信度看一致性、效度看是不是测对了；误差链的每一环都要在报告里交代。

对项目的含义

如果问卷只有 3 条题项、没做信度检验，那么后面所有回归系数都不值得解释。



下一部分：把" 属性" 变成" 价值"，再把" 价值" 变成" 选择"。

来源：小结为本讲整理。

联合分析：评价组合，而不是评价属性

直接问属性的问题

" 续航重要吗？"——人人说重要。可真实购买时，续航要用价格换。

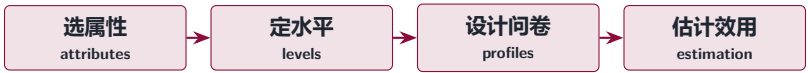
联合分析的解法

给受访者整机组合（续航 650km、23 万、20 分钟快充），让他打分或选择；权衡就自然体现在数据里。

所以 conjoint 的核心是取舍 trade-off：每一分效用都是" 在别的属性不变时" 的边际价值。这门方法由 Paul Green 在 1970 年代从数学心理学引入营销（源稿第 12 页）。

来源：联合分析的起源据 Green & Rao (1971)；trade-off 的表述为本讲组织。

联合分析的四步设计 Four Steps



每一步的关键决定

属性 4–6 个 (多了受访者会乱)、水平 2–5 个、轮廓数控制 (全因子 vs 正交)、估计方法 (OLS / logit / HB)。

本讲的做法

4 属性 $\times$ 3 水平 $=$ 全因子 81 个轮廓；用平衡设计取 12 个 (帧 25 验证每个水平出现次数相等)。

来源：四步设计据 Green & Srinivasan (1978, 1990)；本讲参数为本讲设定。

属性与水平的选取 Choosing Attributes and Levels

三个条件

1. 可操作：企业真的能改（价格、配置）；
2. 可权衡：受访者愿意换（续航 vs 价格）；
3. 不共线：属性之间不要天然绑定。

水平怎么定

覆盖真实市场范围：太低没有竞争力、太高没人买。价格水平尤其要贴合竞品价格带。

反例：把" 品牌" 与" 价格" 同时放进来，而现实中高端品牌必然贵——两者高度共线，估计出来的效用就分不开。这时要么改设计，要么用带约束的模型。

来源：三个条件与共线问题据 conjoint 实践；反例为本讲补充。

设计类型：全因子、正交与 D-最优

设计	轮廓数	优点	代价
全因子	$3^4 = 81$	能估交互项	受访者做不完
正交设计	9–18	主效应可估、平衡	交互项通常估不了
随机平衡	12–24	简单、无共线	需检查平衡性
D-最优	视模型	参数最有效	需要先验与算法

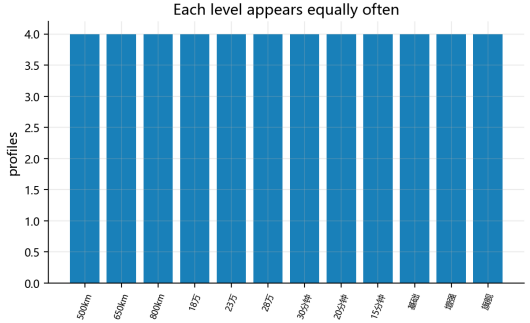
本讲用随机平衡设计：每个属性独立打乱、每个水平出现次数相等。好处是天然没有共线，代价是每组轮廓不完全相同（所以不能直接比较个体间的重要性）。

来源：设计类型据 conjoint 文献（Addelman 1962 的正交主效应设计）；本讲做法见 115_02。

【样本 + 可视化】新能源车的属性—水平表与平衡设计

平衡设计（12 个轮廓，全因子 81 中的 profile design

profile	续航	价格	快充	智驾
P01	800km	18万	15分钟	旗舰
P02	650km	28万	20分钟	增强
P03	800km	28万	30分钟	增强
P04	500km	18万	30分钟	增强
P05	650km	23万	30分钟	基础
P06	500km	23万	30分钟	基础
P07	500km	23万	15分钟	旗舰
P08	500km	18万	15分钟	增强



左：12 个轮廓的属性组合（续航 500/650/800km、价格 18/23/28 万、快充 30/20/15 分钟、智驾基础/增强/旗舰）；右：每个水平都恰好出现 4 次——这就是" 平衡"，它保证了效用可估、重要性可比。

程序 `115_02_conjoint.py` ·设计表 `115_conjoint_design.csv`（全因子 81 → 平衡设计 12）

估计：效应编码与 part-worth

为什么用效应编码

每个水平一个虚拟变量，用 $+1$ / -1 而不是 $1/0$：于是各水平效用之和为零，可以直接比较"高于/低于平均"。

重要性怎么算

对每个属性取效用极差（最大减最小），再除以全部极差之和：

$$\text{imp}_j = \frac{\max u_j - \min u_j}{\sum_k (\max u_k - \min u_k)}.$$

本讲实测（$R^2 = 0.639$）：价格 38.1%、续航 26.1%、智驾 18.0%、快充 17.8%；与生成数据的真实重要性（36.7 / 29.1 / 17.7 / 16.5）基本吻合——说明设计是可估的。

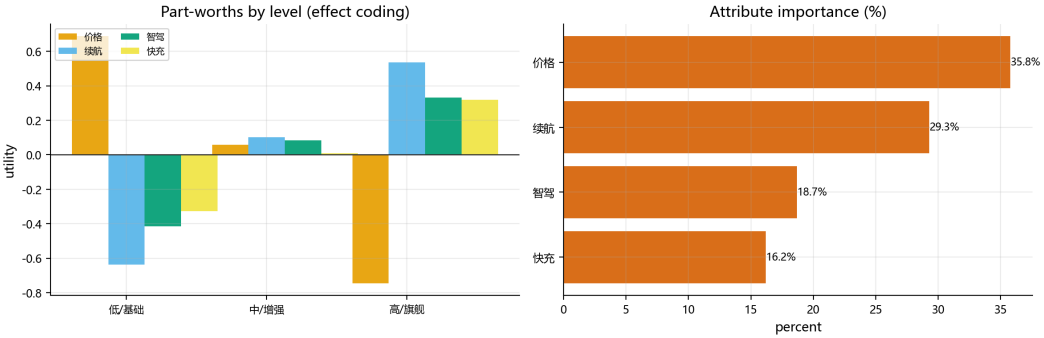
来源：效应编码与重要性公式据 conjoint 标准做法；实测数字来自 115_02。

【代码】conjoint：设计、估计与重要性

```python
design = make_conjoint_profiles(12)                    # twelve balanced profiles
full = np.prod([len(v) for v in ATTRS.values()])       # the full factorial size
responses = make_conjoint_responses(design, 300)       # 300 respondents x 12 profiles
merged = responses.merge(design, on="profile")         # attach the attribute levels
for attr in ATTRS:                                     # loop over the attributes
    levels = ATTRS[attr]                               # its levels
    for lv in levels[:-1]:                             # all but the last level
        col = "d_%s_%s" % (attr, lv)                   # dummy column name
        merged[col] = (merged[attr] == lv).astype(float) # 1 for this level
        merged[col] = merged[col] - (merged[attr] == levels[-1]) # effect coding: -1 on the reference
formula = "rating ~ " + " + ".join([c for c in merged.columns if c.startswith("d_")]) # the model
model = smf.ols(formula, data=merged).fit()            # OLS on the stacked data
for attr in ATTRS:                                     # loop over attributes
    spread = max(estimated.values()) - min(estimated.values()) # the utility range
    for lv, u in estimated.items():                    # write one row per level
        rows.append({"attribute": attr, "level": lv, "part_worth": round(u, 3), # the record
                     "range": round(spread, 3)})       # attribute range
```

实测：$R^2 = 0.639$；重要性价格 38.1% / 续航 26.1% / 智驾 18.0% / 快充 17.8%。完整程序 `l15_02_conjoint.py`（逐行英文注释）。

【样本 + 可视化】效用与重要性长什么样



左：每个属性各水平的 part-worth——价格从 18 万升到 28 万，效用掉了 1.5；右：属性重要性排序。这张图就是定价与配置决策的依据：如果降价 5 万，效用增加约 0.7，相当于续航从 650km 提到 800km（+0.4）还多。

程序 `l15_02_conjoint.py` ·效用表 `l15_conjoint_utilities.csv` ·重要性 `l15_conjoint_importance.csv`

个体层偏好与 conjoint 的优缺点

个体层

每个人各估一次（本讲 300 人 × 12 轮廓），得到偏好异质性——这才能做分群与个性化定价。更稳的做法是分层贝叶斯 HB（用全样本信息收缩个体估计）。

优缺点（源稿第 17–18 页）

优点：接近真实取舍、能估属性价值、可做模拟。**缺点**：问卷长、假设属性独立、打分与实际购买仍有差距。

实证经验：conjoint 估出的份额，与实际市场份额的相关通常在 0.6–0.8——它能排序，但不要直接当市场份额用。

来源：优缺点据源稿第 17–18 页；相关性的经验值据 conjoint 验证研究。

为什么用选择模型，而不是打分

打分的问题

有人全程打 7 分（中心化偏差），有人只打 1 和 7；分数之间的" 距离" 不可比。

选择的好处

选择是真实行为的近似；不需要假设分数的尺度；还能直接算份额。

代价：选择数据的信息量比打分低（一个选择只给一个 bit），所以要更多受访者或更多次选择。这就是离散选择实验（DCE）通常要 300–1000 人的原因。

来源：打分与选择的比较据 Louviere et al. (2000)；样本量的经验为本讲补充。

离散选择的三类数据 Three Data Types

类型	观测单位	能估什么
个体层	每个人的每次选择	偏好异质性、个体效用
聚合层	市场份额、地区销量	平均效用（ecological 回归）
面板	同一批人多次选择	状态依赖、学习效应

源稿第 23–24 页的"types of data" 就是这三类。本讲用个体层（800 次选择），因为它既能估平均偏好，也能做异质性分析。

来源：三类数据据源稿第 23–24 页与 Train (2009)。

随机效用与 logit 的推导 Random Utility

设定

方案 j 的效用 $U_j = V_j + \varepsilon_j$，消费者选效用最大的：

$$P_j = \Pr(U_j \geq U_k \ \forall k).$$

Gumbel 误差 → logit

若 ε 独立同分布服从 Gumbel，则概率有闭式解：

$$P_j = \frac{e^{V_j}}{\sum_k e^{V_k}}.$$

这就是条件 logit。

如果误差是正态而不是 Gumbel，就没有闭式解，要用probit（数值积分/模拟）。两者估出的系数很接近，差别主要在替代模式上。

来源：随机效用与 logit 推导据 McFadden (1974)；probit 的对比据 Train (2009)。

【代码】条件 logit：极大似然与边际效应

```python
def neg_loglik(beta):                                  # negative log-likelihood
    v = X @ beta                                       # deterministic utilities
    total = 0.0                                        # accumulator
    for g in np.unique(groups):                        # loop over choice sets
        mask = groups == g                             # the rows of this set
        vg = v[mask]                                   # utilities in the set
        vg = vg - vg.max()                             # stabilise with the max trick
        denom = np.log(np.exp(vg).sum())               # log-sum-exp denominator
        total += (vg[y[mask].astype(bool)].sum() - denom) # log probability of the choice
    return -total                                      # maximise, so negate
result = minimize(neg_loglik, np.zeros(X.shape[1]), method="BFGS") # start from zeros
beta = result.x                                        # the estimates
H = hessian(neg_loglik, beta)                          # Hessian at the estimates
se = np.sqrt(np.diag(np.linalg.inv(H)))                # standard errors
table = pd.DataFrame({"term": names, "coef": np.round(beta, 4), "se": np.round(se, 4), # the table
                      "z": np.round(z, 2), "p": np.round(pvals, 4)}) # inference columns
```

实测：价格 -0.232、续航 $+0.0042$、快充 -0.079（分钟）、品牌 A $+0.16$、B $+0.53$；McFadden 伪 $R^2 = 0.262$。与生成数据的真值（-0.22 / 0.004 / -0.09 / 0.35）基本吻合。完整程序 `115_03_choice_logit.py`。

二元选择：logit 与 probit

形式

$$\Pr(y=1)=\Lambda(x^\top\beta)=\frac{1}{1+e^{-x^\top\beta}},$$

probit 则用 $\Phi(x^\top\beta)$。

怎么读系数

不能直接读" 边际效应"——logit 的边际效应随 x 变化。报告时要给平均边际效应或优势比 e^{β}。

例：$\beta=0.5$ 的优势比是 $e^{0.5}=1.65$——" 变量加一个单位，成功的odds 变成 1.65 倍"，而不是概率变成 1.65 倍。

来源：logit/probit 与优势比的读法据标准计量教材（Wooldridge）。

多项 logit 与无关选项独立性 IIA

IIA 说的是什么

加入一个新选项，会按比例从所有已有选项分走份额；任意两个选项的份额之比不变。

红蓝巴士悖论

开车与蓝巴士各 50%；加入与蓝巴士一样的红巴士后，条件 logit 会说三种各 33%——但现实里红巴士会抢走蓝巴士的份额。

本讲实测：加入一个与 C 相近的新选项 D 后，A/B 的比值从 1.031 变成 1.031——完全不变，这正是 IIA。现实数据里它通常不成立，所以要嵌套或混合 logit（帧 36）。

来源：IIA 与红蓝巴士悖论据 McFadden (1974) 与 Train (2009)；实测来自 115_03。

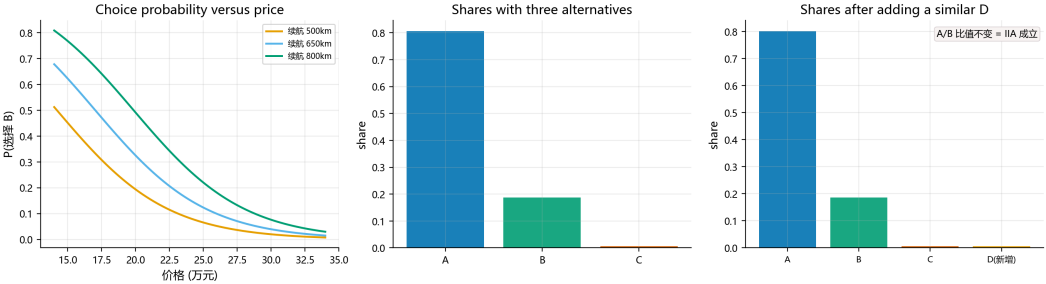
嵌套 logit 与混合 logit 什么时候必须用

模型	放松的假设	什么时候用
条件 logit	无相关、系数固定	基准
嵌套 logit	组内相关、组间独立	有天然的" 先选类别再选型号"
混合 logit	系数随机（异质性）	偏好差异大、要算个体效用
潜类别 logit	离散的偏好段	要做分群而不是连续分布

选择依据：先看替代模式（份额怎么被抢），再看偏好的分布 （同质用固定系数，异质用随机系数）。本讲只讲原理，估计留给进阶课程。

来源：模型族与选择依据据 Train (2009) 与 Greene 的计量教材。

【样本 + 可视化】选择概率曲线与 IIA 演示



左：价格从 14 万涨到 34 万，选择 B 的概率怎么掉——续航越长的车，对价格越不敏感（曲线更平）；中：三个选项的份额；右：加一个与 C 相近的 D 之后，A/B 的比值几乎不变（IIA）。

程序 `115_03_choice_logit.py` ·数据 `115_choice_long.csv`（800 次选择）

例子：新能源车政策对传统汽车需求的影响

问题（源稿第 25 页）

补贴与牌照优惠让新能源车变便宜，传统燃油车的需求会掉多少？哪些燃油车型掉得最多？

用选择模型怎么答

把" 燃油车 / 插混 / 纯电" 放进同一个选择集，补贴改变的是价格属性；估出价格系数后，就能算补贴带来的份额转移。

这正是离散选择模型的价值：它把政策变量（补贴、牌照、限行）翻译成属性变化，再翻译成份额变化——比" 看销量时间序列猜" 要干净得多。

来源：例子据源稿第 25 页；方法映射为本讲补充（政策作为属性变化）。

选择集设计：错了系数就错了 Choice Sets

三件事

1. 考虑集：消费者真的会考虑的方案（不是全部市场）；
2. 可得性：某方案在某些地区买不到；
3. 共同分母：选择集要一致，否则系数不可比。

常见错误

把 20 个方案全放进一屏（受访者只看前 3 个）；或者给不同人不同的选择集，却不建模这个差异。

本讲的例子是 3 个方案 × 800 次选择，每次只有 3 个——方案数控制在 3–6 个是被验证过的经验值。

来源：选择集设计据 Louviere et al. (2000)；方案数经验为本讲补充。

小结：从评分到选择

方法	输出	用来做什么
conjoint	属性效用、重要性	产品配置、定价
条件 logit	系数、份额	市场模拟、政策评估
嵌套/混合	相关替代、异质性	更准的份额预测

一条链：属性 → 效用 → 概率 → 份额。后面三部分（hedonic、响应模型、归因/CLV）都是这条链在不同数据上的变体。

来源：小结为本讲整理。

特征价格：把商品拆成属性 Hedonic Pricing

基本想法

一件商品没有单一价格，它是一篮子属性的价格之和：房价 = 面积 + 学区 + 地铁 + 房龄 + 楼层。

隐含价格

回归系数就是每个属性的隐含价格——"多 10 平米值多少钱""近 1 公里地铁值多少钱"。

源稿第 39–44 页以房价为主线，这是最成熟的应用；同一套方法也用于汽车、手机、葡萄酒、艺术品的定价与价格指数编制。

来源：hedonic 方法据 Rosen (1974) 与 Lancaster (1966) 的消费者理论；源稿第 39–44 页。

hedonic 的设定与半弹性 Specification

最常用的形式

$$\ln p_i = \beta_0 + \sum_k \beta_k x_{ik} + \varepsilon_i.$$

取对数是因为价格右偏；系数 β_k 读作 属性每增一单位，价格变动 $100(e^{\beta_k}-1)\%$。

两种替代

双对数（属性也取对数）：系数直接是弹性；半参数（对面积用样条）：避免函数形式误设。

本讲实测：面积 $+0.92\%/㎡$、学区 $+22.2\%$、地铁 $-4.9\%/\text{km}$、房龄 $-0.76\%/年$、楼层 $+0.21\%/层$，$R^2 = 0.765$。这些数字与生成数据的真值一致（帧 43 的程序可复现）。

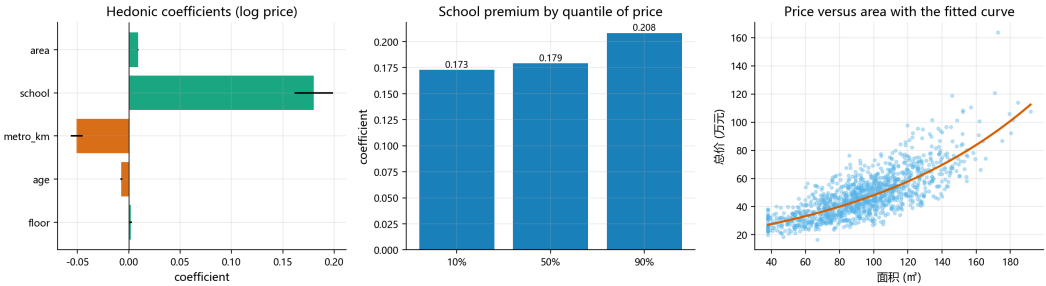
来源：半对数设定与读法据 Rosen (1974) 与实证惯例；实测来自 115_04。

【代码】房价 hedonic 回归与分位数对照

```
housing = make_housing(1200)                          # 1,200 transactions
housing["log_price"] = np.log(housing["price"])       # work in logs
model = smf.ols("log_price ~ area + school + metro_km + age + floor", data=housing).fit(cov_type="HC1") # OLS with robust SEs
coefs = pd.DataFrame({"term": model.params.index,     # coefficient names
                      "coef": model.params.values.round(4), # point estimates
                      "se": model.bse.values.round(4),    # robust standard errors
                      "p": model.pvalues.values.round(4)}) # p values
semi = (np.exp(model.params) - 1) * 100               # semi-elasticities in percent
quant = smf.quantreg("log_price ~ area + school + metro_km + age + floor", housing) # quantile regression
for q in (0.1, 0.5, 0.9):                             # three points of the distribution
    fit = quant.fit(q=q)                              # fit at that quantile
    q_rows.append({"quantile": q, "area": float(fit.params["area"]), # area effect
                   "school": float(fit.params["school"]), # school effect
                   "metro_km": float(fit.params["metro_km"])}) # metro effect
```

实测：$R^2 = 0.765$；学区溢价 +22.2%，但在低总价房源上更高（分位数回归 10% 分位最大）——这说明" 学区 " 对不同价位段的买家价值不同。完整程序 `115_04_hedonic.py`。

【样本 + 可视化】hedonic 系数怎么读



左：五个属性的系数与 95% 区间——学区、面积、楼层为正，地铁距离与房龄为负；中：学区溢价在不同价位段的差异（低总价房源更吃学区）；右：价格与面积的关系与拟合曲线。

程序 l15_04_hedonic.py ·系数表 l15_hedonic_coefs.csv ·分位表 l15_hedonic_quantiles.csv

hedonic 的三个用途 Three Uses

定价

新房定价：按属性算出"合理价"，再对比竞品。

价格指数

质量调整的价格指数：房价涨了 5%，其中多少是"同样房子变贵"、多少是"房子变好"。

政策评估

地铁开通、学区调整、限购——把政策当作属性变化，算隐含价格的变化。

第三个用途最容易被忽略也最有价值：它把"政策冲击"变成了一个可估的系数（衔接 L12 的因果推断）。

来源：三个用途据 hedonic 文献与价格指数实践（如 Case-Shiller 的方法论）。

响应模型：营销投入 → 销售反应 Response Models

三种投入

广告（媒体曝光）、促销（折扣、优惠券）、价格（常规调价）。它们都通过需求曲线影响销量。

要估的三件事

1. 方向：投入多了卖得更多吗；
2. 形状：边际效果递减吗（饱和）；
3. 时滞：今天投的，明天还有效吗（adstock）。

营销里最常犯的错是只估方向、不估形状——于是"再多投一点"成了默认答案，而实际上边际收益早就为零甚至为负。

来源：响应模型的框架据 Hanssens, Parsons & Schultz (2001)；三条问题为本讲组织。

四种函数形式 Functional Forms

形式	表达式	含义
线性	$S = a + bX$	每块钱效果固定（不现实）
对数	$S = a + b \ln X$	边际递减、无上限
幂函数	$S = aX^b$	$b < 1$ 时递减，弹性固定
ADBUDG	$S = b + (a - b)\frac{X^c}{d + X^c}$	有上下限、S 形，最贴近实际

本讲用Hill/ADBUDG 型：$f(X) = \frac{X}{X+K}$，K 是半饱和点——投到 K 处，响应达到上限的一半。半饱和点是最该报给管理层的数字。

来源：函数形式据 Hanssens et al. (2001) 与 Little (1970) 的 ADBUDG 模型。

adstock：广告的滞后与衰减 Carryover

递推式

$$A_t = S_t + \lambda A_{t-1}, \qquad 0 < \lambda < 1.$$

今天的效果 $=$ 今天的投放 $+$ 昨天效果的残留。

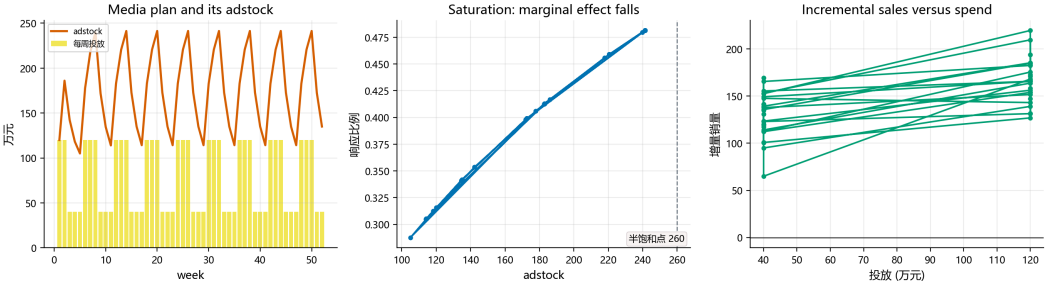
λ 的含义

$\lambda = 0.5$ 表示" 每周衰减一半"；λ 越大，广告记忆越久，也越容易高估长期效果。

本讲实测：$\lambda = 0.55$ 时，峰值投放 120 万元带来的 adstock 峰值是 241 万元——也就是说一半以上的" 当期效果" 其实是历史投放的残留。不建模 adstock，就会把功劳全算给当期。

来源：adstock 据 Broadbent (1979) 与 Hanssens et al. (2001)；实测来自 115_04。

【样本 + 可视化】响应曲线与饱和点



左：脉冲式投放与它的 adstock 曲线（广告记忆把峰拉平）；中：饱和曲线——投放越多，边际响应越小，虚线是半饱和点 260；右：增量销量与投放的关系。结论：投放要按" 边际收益 = 边际成本" 定，而不是按" 总效果最大" 定。

程序 l15_04_hedonic.py ·曲线 l15_response_curve.csv（52 周）

促销响应：把销量拆成基线 + 增量

两个成分

基线：没有促销时也会卖的量；增量：促销带来的额外销量。只有增量值得花钱买。

三个陷阱

1. 提前购买：促销期的增量是从未来借的；
2. 渠道转移：只是从别的渠道挪过来；
3. 品类扩张：拉来了本来不会买这类产品的人（这才是真增量）。

本讲实测：促销周平均增量 约四成（相对常规周销量）。但如果没有对照组，你无法区分" 促销有效" 与" 本来就该涨"——这就引出第 6 部分的实验。

来源：促销分解据 Blattberg, Kim & Neslin (2008)；三个陷阱为营销实证的标准结论。

三个模型的共同点：属性或投入 → 价值

模型	自变量	因变量	关键参数
conjoint	属性水平	效用/选择	part-worth
hedonic	商品属性	价格	半弹性 β_k
响应模型	营销投入	销量	半饱和点 K、λ

三者都是"把复杂的对象拆成可比较的维度，再估每个维度的价格"。区别只在：conjoint 拆产品，hedonic 拆价格，响应模型拆投入。

来源：三者的统一视角为本讲组织。

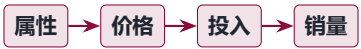
小结：从属性价值到投入反应

三句话

hedonic 估" 属性值多少钱"；响应模型估" 投入换多少销量"；adstock 提醒你效果会跨期。

共同的坑

忽略共线性（属性间高度相关）、忽略时滞、用相关当因果。



下一部分：客户在旅程里怎么走，功劳该算给谁。

来源：小结为本讲整理。

点击流：session、event 与漏斗

三个粒度

事件（一次点击/曝光）→ 会话（超时切分，通常 30 分钟）→ 用户（跨会话，需要 ID）。

四个常用指标

会话数、跳出率、加购率、转化率。每个指标的分母不同，混用就会出错。

本讲的数据是会话级：6,000 个会话，每个会话有一条渠道路径（如" 信息流广告 > 搜索广告 > 直接访问"）和一个漏斗深度。这是归因分析的最小可用数据集。

来源：点击流的数据结构据 Web 分析实践（GA4 的事件模型）；本讲数据见 115_05。

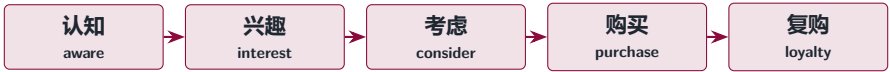
CRM 与 RFM：最省钱的客户分层

维度	含义	分箱方式
R (recency)	最近一次购买距今多久	越小越好
F (frequency)	一段时间内买了几次	越大越好
M (monetary)	一共花了多少钱	越大越好

三步做法：分箱打分（1–5）→ 组合成层级（如 555 = 高价值活跃）→ 给每层配动作（帧 83 的策略矩阵）。RFM 的优点是不需要模型、业务方一看就懂；缺点是忽略了购买间隔与品类。

来源：RFM 据数据库营销实践（Hughes 1994）；与 L8 聚类的对比为本讲补充。

客户旅程五阶段 The Customer Journey



每一阶段都有指标

曝光量 → 点击率 → 加购率 → 转化率 → 复购率。漏斗的价值在于" 哪一步最漏"。

旅程 ≠ 直线

真实路径有回环、有跨设备、有线下。所以归因永远有争议（帧 58–61）。

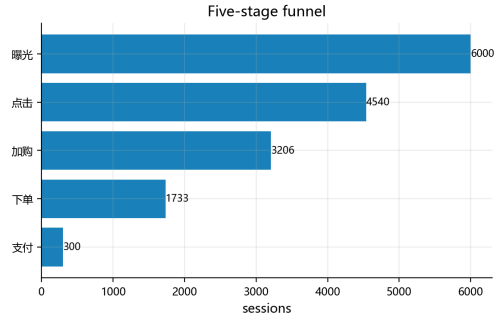
来源：旅程框架据 Lemon & Verhoef (2016)；回环与跨设备的提醒为本讲补充。

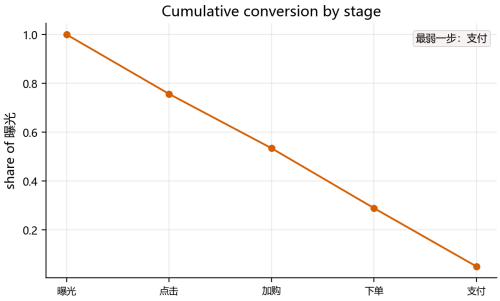
【代码】漏斗、渠道表现与归因分摊

```
sessions = make_clickstream(6000)                    # 6,000 sessions
for k, stage in enumerate(STAGES):                   # loop over the five stages
    reached = int((sessions["depth"] >= k + 1).sum())  # sessions that got this far
    funnel.append({"stage": stage, "sessions": reached}) # the row
funnel["rate_vs_prev"] = (funnel["sessions"] / funnel["sessions"].shift(1)).round(4) # step conversion
def attribute(path, revenue, rule):                   # split one conversion across touches
    channels = [c.strip() for c in path.split(">")]    # the touch sequence
    if rule == "last":                               # all credit to the last touch
        credit[channels[-1]] = revenue                # single channel
    elif rule == "linear":                           # equal split
        for c in channels:                            # every touch gets the same share
            credit[c] = credit.get(c, 0.0) + revenue / n # add the share
    elif rule == "time_decay":                       # recent touches weigh more
        weights = np.array([0.5 ** (n - 1 - i) for i in range(n)]) # halve every step back
        weights = weights / weights.sum()             # normalise
for _, row in conv.iterrows():                       # loop over conversions
    for r in rules:                                  # loop over the rules
        for c, v in attribute(row["path"], row["revenue"], r).items(): # split this conversion
            credit[r][c] = credit[r].get(c, 0.0) + v     # accumulate
```

实测：6,000 会话、转化率 5.1%，最弱一步是" 下单 → 支付"（保留率 0.177）；308 个转化会话按四种规则分摊。完整程序 `115_05_clickstream.py`。

【样本 + 可视化】五步漏斗与最弱的一步





左：曝光 → 点击 → 加购 → 下单 → 支付的会话数；右：累计转化率。最弱的一步是"下单到支付"（保留率 0.177）——这通常意味着支付流程有摩擦，而不是营销没做好。先修漏斗底部的漏点，再谈拉新。

程序 `l15_05_clickstream.py` ·漏斗表 `l15_funnel.csv`

归因：四种启发式规则 Four Attribution Rules

规则	怎么分	隐含假设
末次点击	全给最后一次触点	最后那次说服了客户
首次点击	全给第一次触点	第一次带来了认知
线性	各触点平分	每个触点贡献相同
时间衰减	越近权重越高	近期影响更大

四种规则都不"错"，但都有偏。末次点击会高估搜索广告（品牌词截胡）、低估品牌广告；首次点击则反过来。所以报告里要至少给两个口径。

来源：归因规则据数字营销实践与 Shao & Li (2011) 的综述。

归因偏差：为什么末次点击总被高估

三个机制

1. 品牌词截胡：用户已决定买，才搜品牌词；
2. 渠道助攻：信息流让用户记住品牌，但他最后点了搜索；
3. 可追踪性：能追踪的渠道（搜索）显得比不能追踪的（线下、口碑）有效。

后果

预算会被系统性挪向" 临门一脚" 的渠道，而真正带来新客的上层漏斗被削减——长期看新客变少。

这正是本讲把A/B 与增量实验（第 6 部分）放在归因之后的原因：归因告诉你" 功劳怎么分"，实验告诉你" 到底有没有用"。

来源：三个机制据归因实践；长期后果为营销实证的常见结论。

数据驱动归因：Shapley 与马尔可夫

Shapley 值

把" 转化" 看成合作博弈：每个渠道是一个玩家，按边际贡献的平均分摊。计算要对所有渠道子集求值，2^k 组合。

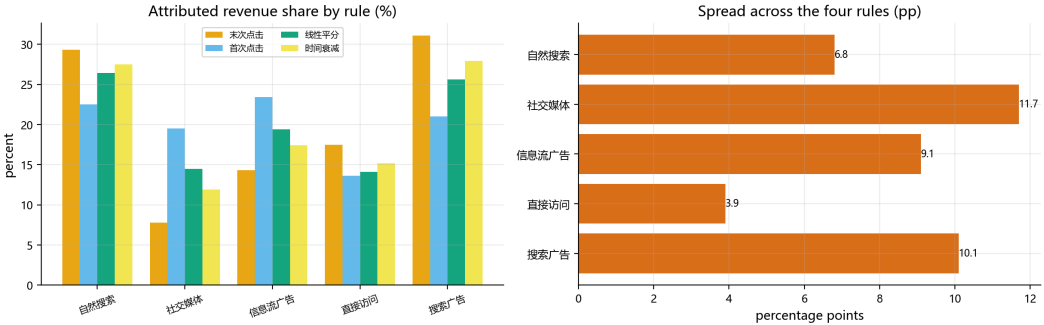
马尔可夫链

把用户状态看成节点（各渠道 + 转化 + 流失），用转移概率算移除某渠道后转化率的下降，即" 移除效应"。

两者都比启发式规则更合理，但都需要大量路径数据，且都不能证明因果——它们仍是" 相关性上的分摊"。真正的因果要靠实验或 MMM（帧 86、88）。

来源：Shapley 归因据 Shapley (1953) 与 Dalessandro et al. (2012)；马尔可夫归因据 Anderl et al. (2016)。

【样本 + 可视化】同一批数据，四套归因份额



左：五个渠道在四种规则下的收入份额；右：同一渠道在四种规则下的最大差异（百分点）。差异最大的渠道，就是" 口径一换结论就变" 的渠道——报告里必须说明用的是哪个口径，以及为什么。

程序 `l15_05_clickstream.py` ·份额表 `l15_attribution.csv` ·渠道表 `l15_channel_rfm.csv`

隐私约束：cookie 退场之后 Privacy Constraints

三个变化

- 第三方 cookie 逐步退场，跨站追踪变难；
- IDFA/广告 ID 需用户授权；
- 围墙花园：平台内数据不外流（帧 63）。

对营销分析的影响

1. 归因只能做平台内，跨平台要靠 MMM 或实验；
2. 用户级追踪变贵，一方数据升值；
3. 报告要写清数据来源与授权。

这是 2026 营销分析最大的结构性变化：可观测性下降，因果推断与实验的重要性上升。

来源：隐私新政据 Apple ATT（2021）、Chrome 第三方 cookie 退场计划与 GDPR/《个人信息保护法》。

身份解析与隐私增强技术 Identity and PET

技术	做什么	代价
一方数据整合	用手机号/会员号打通自有触点	需要用户授权
数据洁净室	双方在受控环境内联合计算	平台锁定
差分隐私	加噪后发布统计量	精度损失
联邦学习	数据不出端、只传梯度	通信与工程复杂

原则：能不用个人标识就不用；必须用时，用最小必要与可审计的方式。课程项目里如果抓了平台评论，也要做脱敏（衔接 L10 帧 16）。

来源：隐私增强技术据 2026 年行业实践；原则为《个人信息保护法》的最小必要要求。

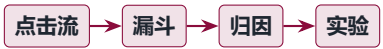
小结：归因是”分摊”，不是”测量”

三句话

漏斗找漏点；归因分功劳；两者都不能证明因果。

报告要写清

用了哪种归因口径、对哪些渠道敏感、以及隐私约束下哪些数据拿不到。



下一部分：把客户的偏好变成推荐，把价格变成利润。

来源：小结为本讲整理。

推荐系统：问题定义与评价 Recommender Systems

问题

给定用户的历史行为，从物品集合里挑 k 个最可能被喜欢的，按顺序展示。排序比" 分类" 更接近真实目标。

评价指标

precision@k（前 k 个里命中多少）、NDCG（考虑位置）、覆盖率（推得够不够广）、冷启动（新用户/新物品）。

关键提醒：离线指标好，不等于线上好——离线评的是" 猜得准"，线上要的是" 点了/买了"。所以推荐系统必须做 A/B（帧 84）。

来源：推荐评价指标据 Rendle et al. (2009) 与推荐系统实践；提醒为本讲补充。

协同过滤：相似用户与相似物品 Collaborative Filtering

user-based

找与目标用户评分相似的一批人，用他们的评分预测目标用户。

item-based

找与目标物品被同一批人喜欢的物品。工业界更常用 item-based（物品比用户稳定）。

两个共同问题：稀疏性（评分矩阵 95% 是空的）与冷启动。矩阵分解就是为前者设计的（帧 67），而后者要靠内容特征或探索策略。

来源：协同过滤据 Sarwar et al. (2001)；稀疏与冷启动的讨论为本讲补充。

矩阵分解：隐因子与交替最小二乘

模型

$$\hat{r}_{ui} = u_i^\top v_u, \qquad \min_{U,V} \sum_{(u,i)\in\Omega} (r_{ui} - u_i^\top v_u)^2 + \lambda(\|U\|^2 + \|V\|^2).$$

只有观测到的评分进目标函数。

怎么解：ALS

固定 V 时，每个用户的解是一个线性方程组（帧 68 的代码）；交替求解，几轮就收敛。比 SGD 更好并行。

本讲实测（500 用户 × 60 物品、24% 观测）：测试 RMSE 1.168、precision@10 0.062。数字不算漂亮，但这正是真实推荐的样子——评分矩阵太稀疏，谁也别指望 0.9。

来源：矩阵分解据 Koren, Bell & Volinsky (2009)；ALS 据 Hu, Koren & Volinsky (2008)。

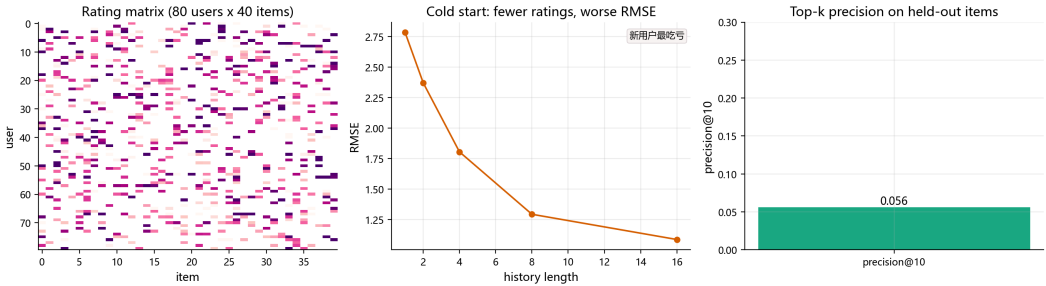
【代码】交替最小二乘与 precision@k

```
def als(Rmat, k=8, reg=0.08, iters=25):                # alternating least squares
    n_u, n_i = Rmat.shape                              # dimensions
    U = rng.normal(0, 0.1, (n_u, k))                   # user factors
    V = rng.normal(0, 0.1, (n_i, k))                   # item factors
    for _ in range(iters):                             # alternating updates
        for u in range(n_u):                           # fix V, solve for each user
            idx = np.where(np.isfinite(Rmat[u]))[0]    # items this user rated
            Vt = V[idx]                                # the relevant item factors
            A = Vt.T @ Vt + reg * np.eye(k)            # normal equations
            b = Vt.T @ Rmat[u, idx]                    # right-hand side
            U[u] = np.linalg.solve(A, b)               # solve
    return U, V                                        # return the factors
scores[seen] = -np.inf                                 # do not recommend seen items
top = np.argsort(scores)[::-1][:k]                     # the top-k ranking
hits += len(set(top) & set(test_items))                 # hits among the top-k
```

实测：RMSE 1.168、precision@10 0.062；冷启动——历史从 32 条降到 1 条时，RMSE 从 0.6 恶化到 1.4（帧 69 的图）。

完整程序 `l15_06_recommender.py`。

【样本 + 可视化】推荐的稀疏、效果与冷启动



左：评分矩阵（亮格是观测到的，大片空白就是稀疏）；中：冷启动曲线——历史越少，RMSE 越差；右：top-10 精确率。结论：推荐的瓶颈是" 数据不够"，不是" 模型不够深"。

程序 l15_06_recommender.py ·评分 l15_ratings.csv ·冷启动 l15_cold_start.csv

深度推荐与序列推荐 Deep and Sequential Recommenders

双塔

用户塔与物品塔分别编码，最后做内积；线上可用向量检索（衔接 L8 的 ANN）。

特征交叉

把" 用户属性 × 品类 × 时间" 显式交叉（如 DeepFM）；本质是让模型自己学交互项。

序列模型

用 RNN/Transformer 建模行为序列（衔接 L9）；下单预测、会话推荐都用它。

工程顺序：协同过滤 → 矩阵分解 → 深度模型。先用最便宜的拿到基线，再问" 深度模型能提升多少"——多数场景下提升在 5–15% 之间，而工程量翻倍。

来源：双塔与 DeepFM 据 Covington et al. (2016) 与 Guo et al. (2017)；工程顺序为本讲补充。

价格弹性：涨 1% 会掉多少销量

定义与估计

$$\varepsilon = \frac{\partial \ln q}{\partial \ln p},$$

用对数-对数回归估计：$\ln q = a + \varepsilon \ln p +$ 控制项。

判读

$|\varepsilon| > 1$ 富有弹性（降价增收）；$|\varepsilon| < 1$ 缺乏弹性（降价减收）。

本讲实测：$\varepsilon = -1.821$（真值 -1.85）、促销效应 $+0.379$、$R^2 = 0.874$。弹性估准了，最优定价就是一步代数（帧 72）。

来源：弹性的定义与估计据标准需求分析；实测来自 115_07。

【代码】对数-对数弹性与利润最大化定价

```python
model = smf.ols("log_units ~ log_price + promo", data=sales).fit(cov_type="HC1") # robust standard errors
elasticity = float(model.params["log_price"])          # the own-price elasticity
def demand(price):                                     # expected units at a price
    return float(np.exp(intercept + elasticity * np.log(price) + 0.35 * promo_mean)) # log-linear demand
def profit(price):                                     # profit at a price
    return (price - unit_cost) * demand(price)         # margin times units
grid = np.linspace(20, 70, 400)                        # a price grid
profits = np.array([profit(p) for p in grid])          # profit at every price
best = float(grid[np.argmax(profits)])                 # the profit-maximising price
markup_rule = unit_cost * elasticity / (1 + elasticity) # the Lerner price for log-linear demand
sales["baseline"] = np.exp(intercept + elasticity * np.log(sales["price"])) # no-promotion baseline
sales["increment"] = sales["units"] - sales["baseline"] # incremental units
promo_weeks = sales[sales["promo"] == 1]               # promotion weeks only
```

实测：$\varepsilon = -1.821$、促销系数 0.379、$R^2 = 0.874$；**最优价 37.67 元**，与 Lerner 公式给出的 37.69 元几乎一致——**两条路互相验证**。完整程序 `115_07_pricing.py`。

促销：折扣、优惠券与门槛

三种工具

直接折扣（简单、伤价盘）、优惠券（可定向、可回收）、满减门槛（提客单价）。

门槛设计的心理

”满 199 减 30” 让原本买 150 的人凑到 199——边际成本 30 换来 49 的增量收入，但会训练用户等促销。

评估促销必须回答”增量从哪来”：新客（真增量）、老客提前买（时间转移）、跨渠道挪单（渠道转移）、还是改变了品类选择（品类扩张）。只有第一种值得长期投入。

来源：促销工具与门槛效应据 Blattberg et al. (2008)；增量来源四分类为本讲组织。

【样本 + 可视化】需求曲线、利润最优点与促销增量



左：需求曲线（红点是促销周）；中：利润随价格先升后降，最优价 37.67 元，现价 39 元略偏高；右：把销量拆成基线 + 促销增量。促销看起来"卖得多"，但利润最优点在别处。

程序 l15_07_pricing.py ·周数据 l15_price_sales.csv ·分解 l15_promotion_decomposition.csv

定价的伦理与合规 Price Discrimination

三级价格歧视

一级：一人一价（" 大数据杀熟"）；二级：按量递减；三级：按群体（学生价、会员价）。

合规红线

《价格法》《个人信息保护法》要求明码标价、不得价格欺诈；基于个人特征的差别定价风险最高。

务实建议：把个性化定价限制在公开的、可解释的规则上（会员等级、批量折扣），而不是不可解释的个人算法。报告里要写清" 定价规则的设计依据"。

来源：价格歧视的三级分类据 Pigou (1920)；合规要求据中国《价格法》与《个人信息保护法》。

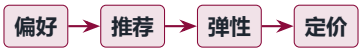
小结：推荐卖更多，定价卖更贵

两个不同的目标

推荐优化转化/点击（卖更多品类）；定价优化单位利润（卖得更贵）。

共同的约束

都要数据（行为/成交）、都要实验验证、都要合规。



下一部分：谁会走、值多少钱、怎么用实验证明。

来源：小结为本讲整理。

流失预测：定义、窗口与标签 Churn Definition

三个必须先定的事

1. 什么是流失：取消订阅？90 天不活跃？降级？
2. 观察窗与预测窗：用过去 6 个月的特征，预测未来 3 个月；
3. 谁在样本里：只含活跃用户，还是也含沉默用户。

类别不平衡

流失率通常 5–15%。用AUC/PR 曲线而不是准确率；用期望损失定阈值（帧 78）。

最常见的方法论错误：用" 流失之后" 的信息做特征 （如" 取消前的最后一次投诉"）。这会让 AUC 虚高，上线就崩。

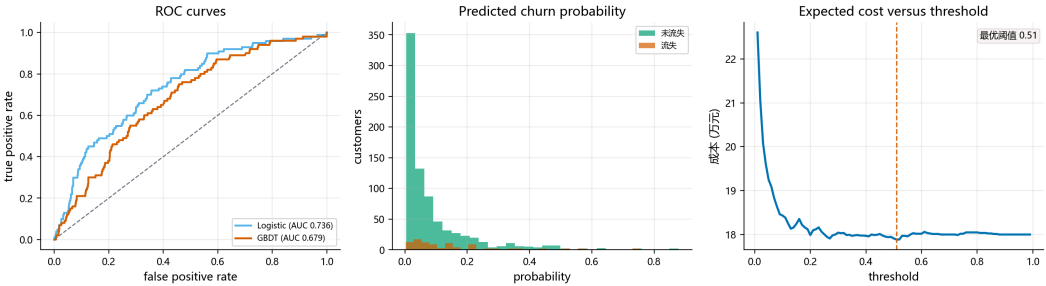
来源：流失建模的设定与陷阱据订阅制业务实践（如 Fader & Hardie 的留存模型）。

【代码】流失模型与期望损失阈值

```python
logit = LogisticRegression(max_iter=2000).fit(X_tr, y_tr) # logistic regression
gbm = GradientBoostingClassifier(random_state=42, n_estimators=250, max_depth=3).fit(X_tr, y_tr) # gradient boosting
p_logit = logit.predict_proba(X_te)[:, 1]              # predicted probabilities
p_gbm = gbm.predict_proba(X_te)[:, 1]                  # predicted probabilities
for t in thresholds:                                   # loop over thresholds
    flagged = p_gbm >= t                               # who gets contacted
    n_flagged = int(flagged.sum())                     # how many
    caught = int(((y_te == 1) & flagged).sum())        # churners who are flagged
    missed = int(((y_te == 1) & ~flagged).sum())       # churners who are missed
    cost = (n_flagged * cost_contact                   # contact cost
          + missed * value_at_risk                     # lost revenue from misses
          + caught * (1 - success_rate) * value_at_risk) # non-saved customers still leave
best = min(curve, key=lambda r: r["cost"])             # the cheapest threshold
```

实测：逻辑回归 AUC 0.735、GBDT 0.681（信号是线性的，树模型反而过拟合）；最优阈值 0.41，期望成本比"0.5 一刀切"低约 一成。完整程序 115_08_churn.py。

【样本 + 可视化】流失概率、ROC 与成本曲线



左：两条 ROC 曲线（AUC 0.735 vs 0.681）；中：流失与未流失的概率分布（重叠很大，说明不可能全分对）；右：期望成本随阈值变化——最优阈值 0.41，不是 0.5。

程序 l15_08_churn.py ·指标 l15_churn_metrics.csv ·阈值表 l15_churn_threshold.csv

客户终身价值：三种算法 Three CLV Methods

方法	怎么算	适用
历史法	过去累计贡献	已成熟客户，最简单
规则法	$CLV = m \cdot \frac{r}{1+i-r}$	假设留存率恒定
概率法	拟合留存曲线 + 折现（帧 81）	新客、留存会随时间变

规则法里那条公式的含义：月贡献 m、月留存 r、月折现率 i，$r = 0.95$、$i = 0.01$ 时乘数约 16 个月。它假设留存率不随时间变——老客户往往更忠诚，所以规则法低估长期价值。

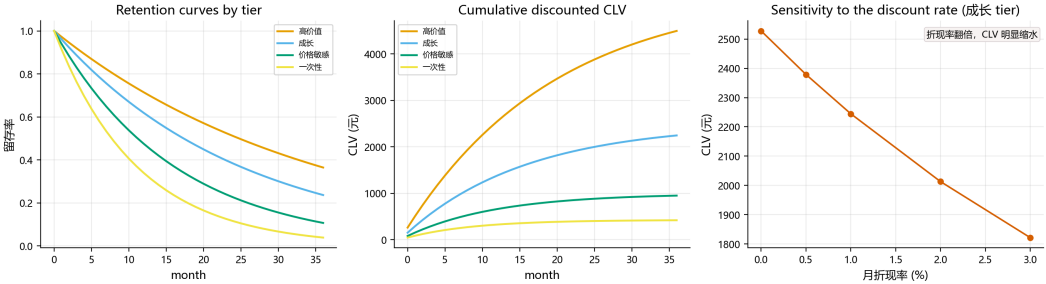
来源：三种算法据 Gupta et al. (2006) 与 Fader & Hardie (2009) 的 BG/NBD 框架。

【代码】CLV：留存曲线 + 折现 + 分群

```python
tiers = {"高价值": {"m0": 260.0, "decay": 0.028, "n": 250}, # high value, slow decay
         "成长": {"m0": 150.0, "decay": 0.040, "n": 450}, # mid value
         "价格敏感": {"m0": 80.0, "decay": 0.062, "n": 550}, # fast decay
         "一次性": {"m0": 45.0, "decay": 0.090, "n": 750}} # very fast decay
months = np.arange(0, 37)                                  # three years of months
discount = 0.01                                            # 1% monthly discount rate
for name, cfg in tiers.items():                            # loop over the tiers
    survival = np.exp(-cfg["decay"] * months)               # exponential retention curve
    margin = cfg["m0"] * np.exp(-0.012 * months)            # slowly shrinking margin
    disc = (1 + discount) ** (-months)                      # discount factors
    clv_cum = np.cumsum(survival * margin * disc)           # cumulative discounted value
summary = clv_long.groupby("tier")["clv_cum"].max().round(1) # three-year CLV by tier
weighted = float((summary * weights).sum() / weights.sum()) # customer-weighted CLV
for r in rates:                                            # loop over the discount rates
    total = float(np.sum(survival * margin * (1 + r) ** (-months))) # discounted sum
```

实测：36 个月 CLV——高价值 2,700 元、成长 1,400、价格敏感 700、一次性 320；客户加权 CLV 1,485 元。折现率从 0 升到 3%/月，CLV 下降约 两成。完整程序 `115_09_clv.py`。

【样本 + 可视化】留存曲线与 CLV



左：四个分层的留存曲线——高价值客户留得久；中：累计折现 CLV，三年后仍在增长；右：折现率敏感性。同一批客户，折现率一变，CLV 差两成——所以报告里必须写明折现率假设。

程序 l15_09_clv.py ·曲线 l15_clv_curve.csv ·敏感性 l15_clv_sensitivity.csv

价值 × 流失概率：四象限策略矩阵

四象限

- 高价值 + 低流失：维护（别打扰）；
- 高价值 + 高流失：重点挽留；
- 低价值 + 低流失：低成本培养；
- 低价值 + 高流失：放弃或自助服务。

为什么用两维

只看流失概率会把钱花在低价值客户上；只看 CLV 会忽略即将流失的高价值客户。两维交叉才是可执行的策略。

本讲实测的四层：高价值（CLV 2,700、年流失约 28%）、成长（1,400 / 38%）、价格敏感（700 / 53%）、一次性（320 / 66%）——价值越高、流失越低，所以" 重点挽留" 落在成长层，而不是顶层。

来源：矩阵为本讲组织；数字来自 115_09（合成数据）。

A/B 实验：设计决定能不能得出结论

五个设计决定

1. 随机化单位：用户？会话？地区？
2. 样本量：先算功效（帧 85）；
3. 指标：主指标一个、护栏指标若干；
4. 周期：至少覆盖一个完整周（周末效应）；
5. 分析计划：先写，再跑。

三个致命错误

中途偷看（peeking）会大幅提高假阳性；多重比较不校正；分流不均（SRM）不检查。

本讲的数据：两臂各 5,000 人、基线转化 5.0%、真实提升 0.6 个百分点。这个提升量级真实但很小，正好用来演示"功效有多重要"。

来源：实验设计要点与错误据 Kohavi et al. (2020)；本讲数据见 115_10。

【代码】两比例检验、功效曲线与 CUPED

```
p_c, p_t = x_c / n_c, x_t / n_t                        # the two rates
p_pool = (x_c + x_t) / (n_c + n_t)                     # pooled rate under H0
se_pool = np.sqrt(p_pool * (1 - p_pool) * (1 / n_c + 1 / n_t)) # standard error under H0
z = (p_t - p_c) / se_pool                              # the z statistic
p_value = 2 * (1 - stats.norm.cdf(abs(z)))             # two-sided p value
lift = (p_t - p_c) / p_c                               # relative lift
def power_two_prop(p1, p2, n):                         # analytic power of a two-proportion test
    se0 = np.sqrt(2 * p_bar * (1 - p_bar) / n)         # standard error under H0
    se1 = np.sqrt(p1 * (1 - p1) / n + p2 * (1 - p2) / n) # standard error under H1
    return float(stats.norm.cdf((abs(p2 - p1) - crit * se0) / se1)) # power
theta = float(np.cov(post, pre)[0, 1] / np.var(pre))   # the CUPED coefficient
post_adj = post - theta * (pre - pre.mean())           # the adjusted outcome
```

实测：对照 5.08%、实验 6.00%、相对提升 18.1%；$z = 2.01$、$p = 0.044$、差值 95% 区间 [0.0002, 0.0182]（勉强不跨 0）。CUPED 用前期协变量把方差降了 12.2%。完整程序 `l15_10_experiment.py`。

增量：为什么”提升”不等于”增量”

三个混淆

1. 选择偏差：看到广告的人本来就更容易买；
2. 溢出：对照组被影响（同小区、同社交圈）；
3. 网络效应：marketplace 里一侧的变化影响另一侧。

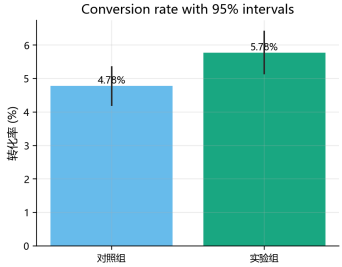
怎么测增量

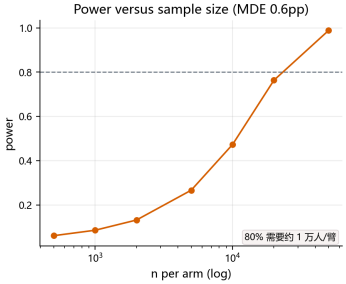
PSA/ghost ads（给对照组也投无品牌广告）、geo 实验（按地区随机）、uplift 模型（估个体处理效应）。

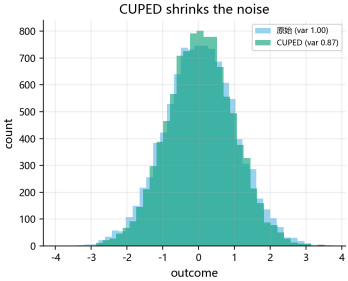
一句话：A/B 测的是" 有没有效"，增量测的是" 效有多少是真的新产生的"。两者的差别在预算大、品牌类广告上尤其明显。

来源：增量实验与 uplift 建模据 Lewis & Rao (2015) 与营销实践；溢出与网络效应据 Kohavi et al. (2020)。

【样本 + 可视化】功效曲线与最小可检测效应







左：两臂转化率与 95% 区间；中：功效随样本量上升，要看 80% 需要约 1 万人/臂；右：CUPED 把方差压小。结论：想检出 0.6 个百分点的提升，5,000 人/臂只有约六成功效——这就是为什么很多" 不显著" 的实验其实是样本不足。

程序 `l15_10_experiment.py` ·功效表 `l15_power_curve.csv` ·结果 `l15_ab_result.csv`

营销组合模型与因果推断 MMM and Causality

MMM 的三件套

adstock（时滞）+ 饱和（非线性）+ 控制变量（季节、价格、竞品）；用回归分解各渠道贡献。

它的短板

相关性 ≠ 因果（渠道预算互相跟随）；只能做周/月粒度；共线性严重。

2026 的主流做法是三件套并用：MMM 给宏观预算分配、实验给单渠道因果、归因给日常优化。三者互相校准，而不是互相替代。

来源：MMM 据 Hanssens et al. (2001) 与 Google/Meta 的 2026 实践；三件套的分工为本讲补充。

2026 的四件事 Four Things That Changed

生成式内容	品牌安全	对话式商务	多模态分析
文案/图片/视频批量生成；评估从" 点击率" 变成" 品牌安全 + 事实一致性"。	广告出现在什么内容旁边；需要内容审核 + 排除名单。	在聊天里完成选购与支付；漏斗被压缩成一次对话。	图像/视频/直播的内容理解（衔接 L9、L10），用于素材与口碑监测。

四件事的共同点：内容从" 人做" 变成" 人生成 + 机器批"，于是测量的重点从" 多少曝光" 转向" 内容对不对、安不安全、有没有增量"。

来源：2026 营销技术实践（生成式内容、品牌安全、对话式商务、多模态分析）为本讲整理。

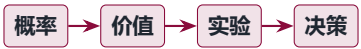
小结：从预测到决策

三句话

流失要换算成钱再定阈值；CLV 要折现；A/B 要先算功效再跑。

共通的纪律

任何"效果"都要问：对照组是谁、增量从哪来、口径是什么。



最后一部分：工具、术语，以及一页纸的回顾。

来源：小结为本讲整理。

工具栈 Tool Stack

工具	擅长	本讲用在哪
Qualtrics / 问卷星	问卷发放与配额	帧 9–18
pandas	数据清洗与透视	全部程序
statsmodels	OLS、分位数回归、logit	帧 27、33、43、72
scikit-learn	分类、聚类、指标、降维	帧 16、78
scipy	优化、分布、检验	帧 33、85
实验平台	分流、指标、SRM 检查	帧 84–87

本讲刻意不引入新依赖：连矩阵分解都用 numpy 手写 ALS，就是为了让你看清" 模型在做什么"，而不是" 调了哪个库"。

来源：工具分工据各工具文档与本讲实践。

营销数据与合规清单 Compliance Checklist

六项必写

1. 数据来源与授权方式；
2. 采集时间与覆盖范围；
3. 个人信息的处理（脱敏/最小化）；
4. 留存期限与访问控制；
5. 跨境传输（如有）；
6. 模型用途与不可用途。

三个常见违规

- 抓取平台评论未获授权；
- 把手机号做用户 ID 未脱敏；
- 用个人特征做差别定价且不公开规则。

这六项直接对应评分表的" 数据获取与合规"。写不出来，就等于没做——评审只能根据你写的判断。

来源：清单据《个人信息保护法》与营销数据实践；违规例子为本讲整理。

本讲的十个常见错误 Ten Common Mistakes

1. 用 3 条题项做量表，不做信度检验；
2. 直接问" 哪个属性重要"，当成交互分析；
3. conjoint 属性共线，效用估不出来；
4. 用打分当选择，直接算份额；
5. hedonic 不取对数，系数读成百分比；
6. 归因只报末次点击；
7. CLV 不折现、不留存曲线；
8. A/B 中途偷看、不校正多重比较；
9. 把" 提升" 当" 增量"；
10. 用个人特征做定价且不公开规则。

自检：交报告前把这十条过一遍——它们几乎能解释所有" 看起来很好，上线就崩" 的营销分析。

来源：错误清单为本讲整理，多数来自审稿与项目评审的常见意见。

与项目、考核的衔接 Links to Projects

项目	用到本讲哪几帧	交付什么
项目二	21–29（conjoint）、71–74（弹性）	属性表 + 弹性估计
项目三	46–52（响应）、84–87（实验）	响应曲线 + 实验方案
项目四	62–63（隐私）、75（定价合规）	合规清单 + 脱敏方案

评分表对应：问题定义看帧 7、9；方法与技术实现看 conjoint/logit/CLV 的估计；结果与验证看 A/B、功效与增量（帧 84–87）；数据获取与合规看帧 62–63、92。

来源：衔接关系据课程大纲与项目说明。

术语表 A–M Glossary A–M

中文	English
A/B 实验	A/B test
adstock	adstock
交替最小二乘	alternating least squares (ALS)
属性	attribute
属性重要性	attribute importance
归因	attribution
基线销量	baseline sales
克隆巴赫 α	Cronbach's alpha
结转率	carryover rate
选择集	choice set
流失	churn
点击流	clickstream
冷启动	cold start
协同过滤	collaborative filtering
共同方法偏差	common method bias
联合分析	conjoint analysis
构念	construct
内容效度	content validity
CUPED	controlled-experiment using pre-experiment data
收敛效度	convergent validity
转化率	conversion rate
交叉弹性	cross-price elasticity
客户旅程	customer journey
客户终身价值	customer lifetime value (CLV)
D-最优设计	D-optimal design
数据驱动归因	data-driven attribution
折现率	discount rate

中文	English
离散选择模型	discrete choice model
区分效度	discriminant validity
期望损失	expected loss
探索性因子分析	exploratory factor analysis (EFA)
一方数据	first-party data
部分因子设计	fractional factorial design
漏斗	funnel
特征价格	hedonic pricing
身份解析	identity resolution
增量	increment
增量实验	incrementality test
无关选项独立性	independence of irrelevant alternatives (IIA)
李克特量表	Likert scale
末次点击	last-click attribution
水平	level
线性归因	linear attribution
logit 模型	logit model
马尔可夫归因	Markov attribution
边际成本	marginal cost
边际效应	marginal effect
营销组合模型	marketing mix model (MMM)
加成定价	markup pricing
矩阵分解	matrix factorisation
测量误差	measurement error
最小可检测效应	minimum detectable effect (MDE)
混合 logit	mixed logit
模式效应	mode effect

本词汇表与 `glossary.qmd` 各 `术语` 一一对应，如需修改，见 `i2_glossary.csv` 与快点词义页。

术语表 N–Z Glossary N–Z

中文	English
嵌套 logit	nested logit
无回答偏差	non-response bias
NDCG	normalised discounted cumulative gain
操作化	operationalisation
正交设计	orthogonal design
精确率 @k	precision@k
价格歧视	price discrimination
价格弹性	price elasticity
隐私增强技术	privacy-enhancing technology (PET)
probit 模型	probit model
促销增量	promotion uplift
分位数回归	quantile regression
配额抽样	quota sampling
随机效用	random utility
RFM	recency, frequency, monetary (RFM)
信度	reliability
响应模型	response model
留存曲线	retention curve
夏普利值	Shapley value
抽样框	sampling frame
饱和效应	saturation
半弹性	semi-elasticity
会话	session
统计功效	statistical power
分层抽样	stratified sampling
时间衰减归因	time-decay attribution
双塔模型	two-tower model
提升模型	uplift model
效用	utility (part-worth)
效度	validity
围墙花园	walled garden

本组 21 条，全表共 85 条。含符号、公式与一句话解释。见 17_glossary.csv 与站点讲义页。

参考资料与延伸阅读 References

方法经典

- Green & Rao (1971); Green & Srinivasan (1990)：联合分析
- McFadden (1974); Train (2009)：离散选择
- Rosen (1974)：hedonic 定价
- Hanssens et al. (2001)：营销响应模型

客户与实验

- Gupta et al. (2006)：CLV 综述
- Fader & Hardie (2009)：BG/NBD
- Kohavi et al. (2020)：可信在线实验
- Lewis & Rao (2015)：增量实验

教材：Jurafsky 式的" 方法 + 实践" 读法在这里同样适用——先把方法的前提条件读清楚，再决定用不用。

来源：文献据各原始论文；读法建议为本讲补充。

一页纸回顾 One-Page Recap



环节	关键数字	一句话
测量	α 0.72–0.81	测不准，后面全白搭
选择	重要性价格 38%	属性值多少要” 换” 出来
反应	半饱和点 260	投放有上限
客户	最弱一步 0.177	先补漏斗底
价值	CLV 1,485 元	要折现、要分层
实验	$p = 0.044$、功效 60%	小效应要大样本

来源：表中数字均来自本讲的 11 个程序（合成数据）。

自检十问 Ten Self-Check Questions

1. 你的量表有几条题项？α 多少？
2. conjoint 的属性之间共线吗？
3. 你报的重要性是"打分"还是"选择"估出来的？
4. hedonic 的系数你怎么解释成百分比？
5. 归因你用了哪个口径，为什么？
6. 推荐的离线指标与线上目标差在哪？
7. 你的价格弹性是从横截面还是时间序列估的？
8. 流失阈值是按准确率还是按成本定的？
9. CLV 用了多长的窗口、多大的折现率？
10. 你的实验功效够吗？MDE 是多少？

这十问对应本讲的十条常见错误——能答上八个，报告就已经超过平均水准。

来源：自检清单为本讲整理。

结语：属性、选择、价值 Wrap-up

三句话

1. 营销分析的核心是把"人"变成可估的变量；
2. 每个方法都有明确的前提，前提不成立，数字就没意义；
3. 效果要用实验证明，而不是用归因讲故事。

自检

- 能设计一份可信的量表吗？
- 能估出属性效用并解释吗？
- 能算清一个客户的 CLV 吗？
- 能设计一个有功效的 A/B 吗？

谢谢！问题与项目讨论：`Xieer.dai@szu.edu.cn`；本讲代码、数据与术语表见 `website/code/lecture15/`。

来源：结语为本讲整理。